

[Courses](#)[Tutorials](#)[Interview Prep](#)[Sign In](#)

Difference Between List and Tuple in Python

Last Updated : 29 May, 2026

Lists and tuples both store collections of data, but differ in mutability, performance and memory usage. [Lists](#) are mutable, allowing modifications, while [tuples](#) are immutable. Choosing between them depends on whether you need to modify the data or prioritize performance and memory efficiency.

Key Differences between List and Tuple

Parameter	List	Tuple
Mutability	Lists are mutable (can be modified).	Tuples are immutable (cannot be modified).
Iteration Speed	Iteration over lists is time-consuming.	Iteration over tuples is faster.
Operations	Lists are better for insertion and deletion operations.	Tuples are more suitable for accessing elements efficiently.
Memory Usage	Lists consume more memory.	Tuples consume less memory.

Parameter	List	Tuple
Built-in Methods	Lists have several built-in methods.	Tuples have fewer built-in methods.
Error Prone	Lists are more prone to unexpected changes and errors.	Tuples, being immutable, are less error-prone.

Mutability Test

List are Mutable

Lists can be modified, meaning their elements can be changed, added or removed after creation.

```
a = [1, 2, 4, 4, 3, 3, 3, 6, 5]

# Modifying an element in the list `a`
a[3] = 77
print(a)
```

× — ▶ — 📄

Output

```
[1, 2, 4, 77, 3, 3, 3, 6, 5]
```

Explanation: Here, we modified the fourth element (index 3) from 4 to 77. Lists allow direct modification of their elements.

Tuples are Immutable

Tuples cannot be modified after creation. Any attempt to change an element will result in an error.

```
b = (0, 1, 2, 3)

# Attempting to modify a tuple
b[0] = 4
print(b)
```

× — ▶ — □

```
Hangup (SIGHUP)
Traceback ...
TypeError: 'tuple' object does not support item assignment
```

Explanation: Tuples do not support item assignment, making them immutable. This prevents unintended modifications.

Performance Comparison

Memory Efficiency Test

Tuples generally consume less memory because they are immutable and do not need extra space for future modifications.

```
import sys

# Creating an empty list and tuple
a = []
b = ()
```

```
# Assigning values
a = ["Geeks", "For", "Geeks"]
b = ("Geeks", "For", "Geeks")

# Checking memory usage
print(sys.getsizeof(a))
print(sys.getsizeof(b))
```

Output

× ▶ 📄

```
88
64
```

Explanation: This code shows that the list takes more memory than the tuple. Tuples, being immutable, require less overhead, making them a better choice for storing fixed data.

Iteration Speed Test

Tuples have a performance advantage in iterations because they are immutable and stored in a contiguous memory block, reducing overhead.

```
import time

# Creating a large list and tuple
a = list(range(100000))
b = tuple(range(100000))

# Timing tuple iteration
```

```
start = time.time_ns()
for i in range(len(b)):
    x = b[i]
end = time.time_ns()
print(end - start)

# Timing list iteration
start = time.time_ns()
for i in range(len(a)):
    x = a[i]
end = time.time_ns()
print(end - start)
```

Output

```
96517901
98218023
```



Explanation: Tuples can be slightly faster to iterate than lists because their immutability allows some internal optimizations, though the difference is generally small.

Operations

Both lists and tuples support a range of operations, including indexing, slicing, concatenation, and more. However, there are some differences between the operations that are available for lists and tuples due to their mutability and immutability, respectively.

Indexing

Both lists and tuples allow you to access individual elements using their index, starting from 0.

```
a = [1, 2, 3] # list
b = (4, 5, 6) # tuple

print(a[0])
print(b[1])
```



Output

```
1
5
```

Slicing

Both lists and tuples allow you to extract a subset of elements using slicing.

```
a = [1, 2, 3, 4, 5]
b = (6, 7, 8, 9, 10)

print(a[1:3])
print(b[:3])
```



Output

```
[2, 3]
(6, 7, 8)
```

Concatenation

Both lists and tuples can be concatenated using the "+" operator.

```
# List Concatenation
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b)

# Tuple Concatenation
a = (7, 8, 9)
b = (10, 11, 12)
print(a + b)
```

× ▶ 📄

Output

```
[1, 2, 3, 4, 5, 6]
(7, 8, 9, 10, 11, 12)
```

List-Specific operations

Only lists support operations that modify their contents:

- Append: Adds an element at the end.
- Extend: Merges another list.
- Remove: Removes the first occurrence of a value.

```
a = [1, 2, 3]

a.append(4)
a.extend([5, 6])
a.remove(2)
```

```
print(a)
```



Output

```
[1, 3, 4, 5, 6]
```

Explanation: `append(4)` method adds the number 4 to the end of the list, making it `[1, 2, 3, 4]`. Next, the `extend([5, 6])` method adds multiple elements `[5, 6]` to the list, updating it to `[1, 2, 3, 4, 5, 6]`. The `remove(2)` method then deletes the first occurrence of 2 from the list, resulting in `[1, 3, 4, 5, 6]`

Uses of Tuples Over Lists

In Python, tuples and lists are both used to store collections of data, but they have some important differences. Here are some situations where you might want to use tuples instead of lists:

- **Immutability:** Tuples are immutable, ensuring data integrity by preventing modifications. Ideal for constants, configurations, and fixed data.
- **Performance:** Tuples are faster and more memory-efficient than lists as they are stored in a single block and don't require extra space for modifications.
- **Memory Efficiency:** Tuples use contiguous memory, while lists need extra space for dynamic resizing, making tuples a better choice for large, unchanging datasets.

Comment

S shubh...

113

Article Tags:

[Python](#)

[Difference Between](#)

Explore

Python Fundamentals

Python Data Structures

Advanced Python

Data Science with Python

Web Development with Python

Python Practice

Python Courses

											Sign In
trial	Data Types	Interview Questions	Examples	Quizzes	DSA Python	Data Science	NumPy	Pandas	Practice	Dja	
				About Us	POTD	Programming	ML and Data	DSA	Corner		
 Corporate & Communications Address:				Legal	Job-A-	Languages	Science	Python	Interview		
A-143, 6th Floor, Sovereign Corporate				Privacy	Thon	DSA	DSA and	Java	Corner		
Tower, Sector- 136, Noida, Uttar				Policy	Blogs	Web	Placements	C++	Aptitude		
Pradesh (201305)				Contact Us	Nation	Technology	Web	Web	Puzzles		
 Registered Address:				Advertise	Skill Up	AI, ML &	Development	Development	GfG 160		
K 061, Tower K, Gulshan Vivante				with us		Data Science	Programming	Data Science	System Design		
Apartment, Sector 137, Noida, Gautam						DevOps	Languages	CS Subjects			

Buddh Nagar, Uttar Pradesh, 201305

GFG

CS Core

DevOps &

Corporate

Subjects

Cloud



Campus

Interview

GATE

Training

Preparation

Trending

Program

Software and

Technologies

Tools

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved